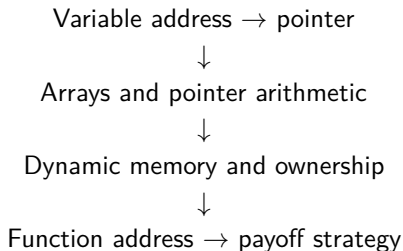


Pointers, Function Pointers, and a Generic CRR Pricer

FRE6883 – Recitation 3

Trace the address, protect the lifetime, choose the payoff

One address lets us change data and behavior



By the end

You can trace a pointer expression, identify unsafe lifetime behavior, and reuse one CRR engine for calls, puts, and digital payoffs.

Every code activity follows an example from Lecture 3, then asks for one prediction or safety check.

A pointer stores where a value lives

```
double dPrice = 21.68;  
double* dPtr = &dPrice;
```

```
std::cout << dPtr;    // an address  
std::cout << *dPtr;   // 21.68
```

`&dPrice`

The address of the variable.

`double* dPtr`

A pointer that may point to a double.

`*dPtr`

Dereference the pointer: access the value at that address.

Keep the two meanings of * separate

In a declaration, * introduces a pointer. In an expression, * follows the stored address.

01 | Complete the address and dereference operations – Problem

GUIDED CODE Pointer basics

Predict before compiling

```
double dPrice = 21.68;
double* dPtr = /* address of dPrice */;

std::cout << /* print the address */ << std::endl;
std::cout << /* print 21.68 */ << std::endl;

/* change dPrice through dPtr */ = 24.50;
std::cout << dPrice << std::endl;
```

Expected final output

The last line prints 24.5. Explain why no assignment to the name dPrice was needed.

01 | Complete the address and dereference operations – Solution

```
double dPrice = 21.68;
double* dPtr = &dPrice;

std::cout << dPtr << std::endl;
std::cout << *dPtr << std::endl;

*dPtr = 24.50;
std::cout << dPrice << std::endl;
```

Alias

dPtr and dPrice give two access paths to the same object.

Type check

A double* must point to a double.

Address output

The hexadecimal-looking address is machine- and run-dependent.

02 | Trace two aliases to one price – Problem

GUIDED CODE Aliasing

Fill the three outputs

```
double price = 100.0;
double* p = &price;
double* q = p;

*p += 5.0;
std::cout << price << " ";           // -----

*q *= 1.10;
std::cout << *p << " ";               // -----
std::cout << (p == q) << "\n";       // -----
```

Question

Are there one, two, or three double objects in this example?

02 | Trace two aliases to one price – Solution

```
*p += 5.0;    // price is 105.0
*q *= 1.10;   // the same price is 115.5

// Output: 105 115.5 1
```

One object

There is one double and two pointer variables. Both pointers store the same address.

Debugging consequence

A write through either alias changes what every other alias observes.

An array name identifies its first element

```
int values[] = {2, 4, 6};  
int* ptr = values;
```

```
values[0]    // 2  
*values      // 2  
ptr[1]       // 4  
*(ptr + 2)   // 6
```

Key equivalence

`values[i]` means `*(values + i)`.

Scaled movement

Adding one moves by one element, not one byte.

No guardrail

Raw array access performs no bounds checking.

03 | Write the same array access four ways – Problem

GUIDED CODE Arrays and pointers

Make every line assign 12 to index 2

```
int values[] = {2, 4, 6};  
int* ptr = values;
```

```
values[/ * ? */] = 12;  
ptr[/ * ? */] = 12;  
*(values + / * ? */) = 12;  
*(ptr + / * ? */) = 12;
```

Constraint

Do not change the operators. Only complete the four indices or offsets.

03 | Write the same array access four ways – Solution

```
values[2] = 12;  
ptr[2] = 12;  
*(values + 2) = 12;  
*(ptr + 2) = 12;
```

The compiler sees the same location

All four expressions target the third `int`. Choose array notation for readability; understand pointer notation so you can trace lower-level code.

Boundary check

`values[3]` compiles, but it accesses outside this three-element array: the behavior is undefined.

Parentheses decide whether we move or add

```
int values[] = {2, 4, 6};  
int* ptr = values;
```

```
*(ptr + 1)  // 4  
*ptr + 1    // 3
```

Move, then dereference

`*(ptr + 1)` selects the next element.

Dereference, then add

`*ptr + 1` reads the first value and adds one to it.

04 | Trace pointer arithmetic without running it – Problem

GUIDED CODE Movement and distance

Complete the outputs

```
int values[] = {2, 4, 6};
int* ptr = values;

std::cout << *(&ptr) << " "; // ____
ptr += 1;
std::cout << *ptr << " ";    // ____
std::cout << ptr - values;    // ____
```

Invariant

At every step, state the index at which `ptr` points before stating the value.

04 | Trace pointer arithmetic without running it – Solution

```
std::cout << *(++ptr) << " "; // ptr -> values[1], prints 4
ptr += 1;                      // ptr -> values[2]
std::cout << *ptr << " ";      // prints 6
std::cout << ptr - values;      // prints 2
```

Pointer subtraction

For two pointers into the same array, subtraction reports the number of elements between them.

Output

4 6 2

Comparing pointers is different from comparing values

```
if (p == q)      // same address?  
if (*p == *q)    // same pointed-to value?
```

Address equality

`p == q` asks whether the two pointers identify the same object.

Value equality

`*p == *q` dereferences both pointers and compares the two values.

05 | Predict address equality and value equality – Problem

GUIDED CODE Pointer comparison

Fill both booleans

```
int x = 7;
int y = 7;
int* p = &x;
int* q = &y;

std::cout << (p == q) << " "; // -----
std::cout << (*p == *q) << "\n"; // -----
```

Explain before revealing

The integers begin with equal values, but they are distinct objects.

05 | Predict address equality and value equality – Solution

```
std::cout << (p == q) << " ";    // 0: distinct addresses  
std::cout << (*p == *q) << "\n"; // 1: equal values
```

Output

0 1

Safety precondition

Only dereference a pointer when it points to a live object. Comparing a pointer with `nullptr` does not dereference it.

A pointer parameter can modify the caller's object

```
void getNum(int* ptr)
{
    std::cin >> *ptr;
}
```

```
int n = 1;
getNum(&n);
```

At the call

`&n` passes the address of the caller's integer.

Inside

`*ptr` accesses that integer.

Effect

The assignment changes `n`, just as a non-const reference parameter would.

06 | Complete call by pointer – Problem

GUIDED CODE Lecture 3 `getNum`

Make the caller observe 9

```
void getNum(int* ptr)
{
    std::cin >> /* pointed-to integer */;
}

int main()
{
    int number = 1;
    getNum(/* address of number */);
    std::cout << number << std::endl;
}
```

Input

If the user enters 9, the program must print 9.

06 | Complete call by pointer – Solution

```
void getNum(int* ptr)
{
    std::cin >> *ptr;
}
```

```
int number = 1;
getNum(&number);
```

Contract

The pointer must not be nullptr; the function dereferences it immediately.

Reference comparison

`void getNum(int& number)` expresses the same required-object relationship more directly.

GUIDED CODE Lecture 3 MySwap

Complete three assignments

```
void MySwap(int* x, int* y)
{
    int temp = /* value pointed to by x */;
    /* object x */ = /* value pointed to by y */;
    /* object y */ = temp;
}
```

```
int a = 2, b = -3;
MySwap(/* two addresses */);
```

Postcondition

After the call, `a == -3` and `b == 2`.

07 | Swap two caller-owned integers – Solution

```
void MySwap(int* x, int* y)
{
    int temp = *x;
    *x = *y;
    *y = temp;
}
```

```
int a = 2, b = -3;
MySwap(&a, &b);
```

Reason about effects

The pointer variables `x` and `y` are local copies of addresses, but dereferencing them reaches the caller's objects.

Read pointer declarations from right to left

Declaration	Can move pointer?	Can change value through it?
<code>double* p</code>	yes	yes
<code>const double* p</code>	yes	no
<code>double* const p = &x</code>	no	yes
<code>const double* const p = &x</code>	no	no

Two different promises

`const` before the pointed-to type protects the value. `const` after `*` fixes the address stored in the pointer.

08 | Decide which operations compile – Problem

GUIDED CODE Pointer to const versus const pointer

Mark valid or invalid

```
double x = 10.0, y = 20.0;
const double* readOnly = &x;
double* const fixed = &x;

readOnly = &y;    // valid / invalid?
*readOnly = 7.0; // valid / invalid?
fixed = &y;       // valid / invalid?
*fixed = 7.0;    // valid / invalid?
```

Reading trick

Ask two separate questions: may the address change, and may the pointed-to value change?

08 | Decide which operations compile – Solution

```
readOnly = &y;    // valid: pointer may move  
*readOnly = 7.0; // invalid: value is protected  
fixed = &y;      // invalid: pointer is fixed  
*fixed = 7.0;    // valid: x becomes 7.0
```

Useful parameter

`const double*` data lets a function read an array while promising not to edit its elements.

Important nuance

The original variable may still be mutable through another non-const access path.

Dynamic memory has an explicit lifetime

`new` / `new[]` $\longrightarrow$ use the live object $\longrightarrow$ `delete` / `delete[]`

Scalar

```
double* p = new double;  
Release with delete p;.
```

Array

```
double* p = new double[n];  
Release with delete[] p;.
```

Ownership question

Every successful allocation needs a clearly responsible owner that releases it exactly once.

GUIDED CODE Dynamic scalar

Complete the lifetime

```
double* price = /* allocate one double */;  
/* assign through the pointer */ = 21.68;  
std::cout << /* pointed-to value */ << std::endl;  
  
/* release scalar memory */;  
price = /* safe empty state */;
```

Expected output

21.68

09 | Allocate, use, and release one price – Solution

```
double* price = new double;  
*price = 21.68;  
std::cout << *price << std::endl;
```

```
delete price;  
price = nullptr;
```

After deletion

The dynamic object no longer exists. Setting the pointer to `nullptr` prevents this variable from continuing to look usable.

It does not repair aliases

Any other pointer that stored the old address is still dangling.

10 | Build a dynamically sized payoff row – Problem

GUIDED CODE Dynamic array

Match every bracket

```
int N = 3;
double* payoff = new double[/* element count */];

for (int i = 0; i <= N; ++i)
{
    payoff[i] = static_cast<double>(i * i);
}

std::cout << payoff[3] << std::endl;
/* release array memory */;
payoff = nullptr;
```

10 | Build a dynamically sized payoff row – Solution

```
int N = 3;
double* payoff = new double[N + 1];

for (int i = 0; i <= N; ++i)
{
    payoff[i] = static_cast<double>(i * i);
}

std::cout << payoff[3] << std::endl; // 9
delete[] payoff;
payoff = nullptr;
```

Pairing rule

`new[]` pairs with `delete[]`; scalar `new` pairs with scalar `delete`.

Leaks, dangling pointers, and invalid dereferences differ

Failure	What happened	Prevention
Memory leak	Last owning address is lost before delete	Preserve ownership; release once
Dangling pointer	Pointer still stores an address after object lifetime ends	Stop using it; set owned pointer to <code>nullptr</code>
Null dereference	Code evaluates <code>*p</code> when <code>p == nullptr</code>	Check the contract before dereferencing
Wrong return	Function returns the address of a local object	Return by value or return live dynamic/caller data

Modern C++ note

Lecture 3 teaches raw ownership directly. In production code, containers and smart pointers usually make the lifetime safer.

11 | Classify four ways to return or modify a number – Problem

GUIDED CODE Lifetime audit from Lecture 3

Safe, unsafe, or null?

```
int* getNum1() { int n = 10; return &n; }
int  getNum2() { int n = 10; return n + 1; }
void getNum3(int* p) { *p += 1; }
int* getNum4() {
    int* p = new int{11};
    delete p;
    p = nullptr;
    return p;
}
```

Task

For each function, state what it returns and whether the caller may safely dereference the result.

11 | Classify four ways to return or modify a number – Solution

Function	Result	Judgment
getNum1	dangling address	unsafe: local <code>n</code> dies on return
getNum2	value 11	safe: return by value
getNum3	no return value	safe when caller passes a valid address
getNum4	<code>nullptr</code>	cannot dereference; the allocated object was deleted

A safe returning pointer must identify live data

That data must belong to the caller, have static lifetime, or remain dynamically allocated with a clear owner.

A function pointer stores which operation to call

```
int addition(int a, int b)
{
    return a + b;
}
```

```
int (*operation)(int, int) = addition;
int result = operation(1, 2);
```

Return type

The pointed-to function returns int.

Parameter list

It accepts two int arguments.

Behavior as data

Assign another compatible function to change the operation without changing the calling code.

12 | Complete a function pointer declaration – Problem

GUIDED CODE Lecture 3 addition example

Match the exact signature

```
int addition(int a, int b)
{
    return a + b;
}
```

```
int (/* pointer syntax */)(int, int) = addition;
int result = /* invoke through pointer */(1, 2);
std::cout << result << std::endl;
```

Why the parentheses matter

Without parentheses around `*operation`, the declaration describes a function returning a pointer, not a pointer to a function.

12 | Complete a function pointer declaration – Solution

```
int addition(int a, int b)
{
    return a + b;
}
```

```
int (*operation)(int, int) = addition;
int result = operation(1, 2);
```

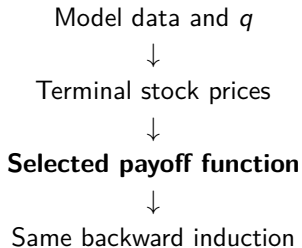
Compatible functions

subtraction, maximum, or any other function with signature `int(int,int)` can be assigned.

Invariant call site

The expression `operation(a,b)` stays unchanged while the selected behavior changes.

One CRR engine should not hard-code one payoff



What varies

At expiry, a call, put, or digital option applies a different function to the same stock value $S(N, i)$.

What stays fixed

`RiskNeutProb`, `CalculateAssetPrice`, the loops, discounting, and array storage do not change.

13 | Add a payoff parameter to PriceByCRR – Problem

GUIDED CODE Function pointer + dynamic array

Complete the changing pieces

```
double* PriceByCRR(double S0, double U, double D,
                  double R, int N, double K,
                  double (*name)(double, double))
{
    double q = RiskNeutProb(U, D, R);
    double* Price = new double[/* size */];
    for (int i = 0; i <= N; ++i)
    {
        double stock = CalculateAssetPrice(S0, U, D, N, i);
        Price[i] = /* call selected payoff */;
    }
    // backward induction follows
}
```

13 | The terminal row calls the selected payoff – Solution A

```
double* PriceByCRR(double S0, double U, double D,  
                  double R, int N, double K,  
                  double (*Payoff)(double, double))  
{  
    double q = RiskNeutProb(U, D, R);  
    double* Price = new double[N + 1];  
    for (int i = 0; i <= N; ++i)  
    {  
        double stock = CalculateAssetPrice(S0, U, D, N, i);  
        Price[i] = Payoff(stock, K);  
    }  
}
```

Single substitution point

Only the terminal payoff evaluation depends on the option type.

13 | Backward induction remains unchanged – Solution B

```
for (int n = N - 1; n >= 0; --n)
{
    for (int i = 0; i <= n; ++i)
    {
        Price[i] =
            (q * Price[i + 1]
             + (1.0 - q) * Price[i]) / R;
    }
}
return Price;
}
```

Ownership transfer

The returned pointer owns an array created with `new[]`. The caller reads `Price[0]` and must eventually call `delete[]`.

14 | Complete call and put payoff functions – Problem

GUIDED CODE Same signature, different rule

Fill both branches

```
double CallPayoff(double z, double K)
{
    if (/* in the money */) return /* payoff */;
    return 0.0;
}
```

```
double PutPayoff(double z, double K)
{
    return (/* in the money */) ? /* payoff */ : 0.0;
}
```

Acceptance cases at $K = 100$

$z = 80$: call 0, put 20; $z = 120$: call 20, put 0.

14 | Complete call and put payoff functions – Solution

```
double CallPayoff(double z, double K)
{
    if (z > K) return z - K;
    return 0.0;
}
```

```
double PutPayoff(double z, double K)
{
    return (z < K) ? (K - z) : 0.0;
}
```

Why one pointer type accepts both

Both functions return double and accept exactly two double parameters.

15 | Price a call and a put without leaking memory – Problem

GUIDED CODE Lecture 3 driver

Complete selection and cleanup

```
double* optionPrice = nullptr;

optionPrice = PriceByCRR(S0, U, D, R, N, K,
                        /* call function */);
std::cout << optionPrice[0] << std::endl;
/* release array */;
optionPrice = nullptr;

optionPrice = PriceByCRR(S0, U, D, R, N, K,
                        /* put function */);
std::cout << optionPrice[0] << std::endl;
/* release array */;
optionPrice = nullptr;
```

15 | Price a call and a put without leaking memory – Solution

```
double* optionPrice = nullptr;

optionPrice = PriceByCRR(S0, U, D, R, N, K,
                        CallPayoff);
std::cout << optionPrice[0] << std::endl;
delete[] optionPrice;
optionPrice = nullptr;

optionPrice = PriceByCRR(S0, U, D, R, N, K,
                        PutPayoff);
std::cout << optionPrice[0] << std::endl;
delete[] optionPrice;
optionPrice = nullptr;
```

Lecture 3 parameters

For $S_0 = 106$, $U = 1.15125$, $D = 0.86862$, $R = 1.00545$, $N = 8$, $K = 100$: call = 21.68, put = 11.43.

Overloading lets one name serve two input shapes

```
int GetInputData(double& S0, double& U,  
                double& D, double& R);
```

```
int GetInputData(int& N, double& K);
```

```
GetInputData(S0, U, D, R); // model overload  
GetInputData(N, K);       // option overload
```

How the compiler chooses

The name is the same, but the number and types of arguments are different. The matching parameter list selects the overload.

Return type is not enough

Two functions cannot be overloaded only by changing their return type.

16 | Add digital options without changing CRR – Problem

GUIDED CODE Lecture 3 extension

Complete two new strategies

```
double DigitCallPayoff(double z, double K)
{
    return (/* condition */) ? 1.0 : 0.0;
}
```

```
double DigitPutPayoff(double z, double K)
{
    return (/* condition */) ? 1.0 : 0.0;
}
```

```
double* digitalCall = PriceByCRR(
    S0, U, D, R, N, K, /* function */);
```

Boundary convention

In the lecture, both digital payoffs return zero when $z = K$.

16 | Add digital options without changing CRR – Solution

```
double DigitCallPayoff(double z, double K)
{
    return (z > K) ? 1.0 : 0.0;
}
```

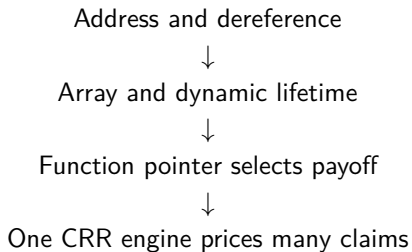
```
double DigitPutPayoff(double z, double K)
{
    return (z < K) ? 1.0 : 0.0;
}
```

```
double* digitalCall = PriceByCRR(
    S0, U, D, R, N, K, DigitCallPayoff);
delete[] digitalCall;
```

Open/closed design

The pricer is open to a new payoff function but closed to edits in its backward-induction algorithm.

The pointer story now forms one complete pricing workflow



Three questions before running code

1. What object or function does this pointer identify?
2. Is that target still alive and compatible?
3. Who owns any dynamic memory and releases it?

Brain teasers combine the Lecture 3 tools

For each challenge

1. Trace the pointers by hand.
2. Complete the missing operations.
3. State the lifetime and bounds invariant.
4. Give one boundary test.

Difficulty ladder

Reverse an array, compact values in place, solve a sorted pair sum, then select a payoff function dynamically.

These can be used as the final class challenge or as reserve practice.

Brainteaser 01 | Reverse an array with two raw pointers – Problem

GUIDED CODE In-place two pointers

Complete movement and swap

```
void Reverse(int* begin, int* end)
{
    while (begin < end)
    {
        /* move end from one-past-last to last */;
        if (begin >= end) break;
        int temp = /* left value */;
        *begin = /* right value */;
        *end = temp;
        /* move begin right */;
    }
}

int a[] = {1, 2, 3, 4, 5};
Reverse(a, a + 5);
```

Target

The array becomes {5,4,3,2,1} in $O(n)$ time and $O(1)$ extra space.

Brain teaser 01 | Reverse an array with two raw pointers – Solution

```
void Reverse(int* begin, int* end)
{
    while (begin < end)
    {
        --end;
        if (begin >= end) break;
        int temp = *begin;
        *begin = *end;
        *end = temp;
        ++begin;
    }
}
```

Invariant

Elements outside the half-open interval $[begin, end)$ are already in their final reversed positions.

Brain teaser 02 | Compact non-zero values in place – Problem

GUIDED CODE Read and write pointers

Return the new logical end

```
int* CompactNonZero(int* begin, int* end)
{
    int* write = begin;
    for (int* read = begin; read != end; ++read)
    {
        if (/* keep this value */)
        {
            /* copy to output position */;
            /* advance output position */;
        }
    }
    return /* one-past-last kept value */;
}
```

Example

{0,5,0,-2,7} begins with {5,-2,7} after compaction; the returned pointer minus begin equals 3.

Brainteaser 02 | Compact non-zero values in place – Solution

```
int* CompactNonZero(int* begin, int* end)
{
    int* write = begin;
    for (int* read = begin; read != end; ++read)
    {
        if (*read != 0)
        {
            *write = *read;
            ++write;
        }
    }
    return write;
}
```

Invariant

[begin, write) contains exactly the non-zero values seen so far, in their original order.

Brainteaser 03 | Find a target sum in a sorted array – Problem

GUIDED CODE Const pointers + linear scan

Move the correct side

```
bool HasPairSum(const int* values, int n, int target)
{
    const int* left = values;
    const int* right = values + n - 1;
    while (left < right)
    {
        int sum = *left + *right;
        if (sum == target) return true;
        if (sum < target) /* move left */;
        else              /* move right */;
    }
    return false;
}
```

Example

{1,2,4,7,11}, target 9 returns true from $2 + 7$.

Brainteaser 03 | Find a target sum in a sorted array – Solution

```
if (sum == target) return true;  
if (sum < target) ++left;  
else             --right;
```

Why moving left works

The array is sorted. If the sum is too small, decreasing the right value cannot help; increase the left value.

Complexity

Each pointer moves inward at most $n - 1$ times: $O(n)$ time and $O(1)$ extra space.

GUIDED CODE Function pointer as a return value

Complete three branches

```
using Payoff = double (*)(double, double);
```

```
Payoff SelectPayoff(char code)
{
    if (code == 'C') return /* call */;
    if (code == 'P') return /* put */;
    return /* no valid function */;
}
```

```
Payoff payoff = SelectPayoff('P');
if (payoff != nullptr)
    std::cout << payoff(80.0, 100.0);
```

Brain teaser 04 | Return the selected payoff function – Solution

```
using Payoff = double (*)(double, double);
```

```
Payoff SelectPayoff(char code)
{
    if (code == 'C') return CallPayoff;
    if (code == 'P') return PutPayoff;
    return nullptr;
}
```

Safe call site

Check the returned pointer before invoking it. With code 'P', `payoff(80,100)` returns 20.

Extension

Add 'D' for a digital call without changing the calling code.

The final test is to explain each star and each delete

Exit ticket

1. What does * mean here?
2. Is the target alive and in bounds?
3. Does const protect the pointer or the value?
4. Who owns the allocation?

Transfer to CRR

The payoff function pointer changes the terminal condition. It does not change the tree mechanics, risk-neutral probability, or backward induction.

If you can answer the four questions before running the program, you can debug pointer code instead of guessing at addresses.